CS6140: Machine Learning — Checkpoint Progress Report

Track 1: replicate and critique a recent paper Name: Rohan Rajendra (Instructor: Virgil Pavlu)

Gradient-Boosted vs. Neural Rankers, and Embeddings as Tree Features

1. Problem, solution, pipeline. Learning to rank: for each search query, sort its candidate documents so the most relevant come first. Tree models (LambdaMART, run with LightGBM) have long been the best at this. Qin et al. (ICLR 2021) propose a neural model, DASALC, and report that it finally matches them.

I rebuild both models, train them on the same public datasets, and compare my results with the paper's. As an extension, on Istella22 (the only dataset with raw text), I test whether relevance scores from a transformer model (ColBERTv2) help LambdaMART when added as extra features.

Pipeline: load each dataset, train on its training split, pick settings on its validation split, then measure on its test split with NDCG, a 0–1 score of how well the top results are ordered (higher is better).

2. Datasets (all downloaded).

Dataset	Queries	Why I use it
MQ2007, MQ2008	1.7K, 0.8K	small; hand check of the math
MSLR-WEB10K, WEB30K	10K, 31K	main benchmark; WEB30K is in the paper
Yahoo Set 1	30K	in the paper
Istella (full)	33K	in the paper
Istella-S	33K	same queries, shorter lists
Istella22	2.2K test	has raw text, for the extension

All are downloaded; all except Istella22 are already used in runs.

3. Hardware and code. **Hardware:** my laptop (Apple M4, 16 GB memory) runs everything except full Istella, which needs more memory and ran on Kaggle's free cloud machines (31 GB). Kaggle's free GPUs will handle the larger neural runs ahead.

Code: everything runs end to end from one git repository: data loading, both models, the NDCG metric, and a by-hand check of the math.

4. Baselines and pilot results. NDCG@5 on each test set, out of 100 (higher is better).

Dataset	Model	Mine	Paper
MSLR-WEB30K	LambdaMART	49.6	49.7
Yahoo Set 1	LambdaMART	74.8	74.2
Istella	LambdaMART	70.4	71.2
MSLR-WEB30K	DASALC	48.5	50.5*
MSLR-WEB10K	LambdaMART	47.1	—
MSLR-WEB10K	DASALC	45.5	—
MSLR-WEB10K	plain network	43.8	—

* The paper's DASALC trained without noise, like mine.

What this means. My LambdaMART reproduces the paper: within 0.1 points on WEB30K and within 1 point on Yahoo and Istella, so the baseline is sound. My DASALC is 2 points below the paper on WEB30K. That is expected for now: mine is a small network (512 units per layer) trained for 16 minutes, while the paper tuned sizes up to 4,096 units. On WEB10K, DASALC beats the same network without attention (45.5 vs. 43.8), as the paper found, but LambdaMART is still ahead of both.

Two caveats. Each of my numbers comes from a single run, so small differences may be noise. And the paper does not say how it scores queries with no relevant document; I leave them out, which is what makes my WEB30K numbers match (counting them as 0 would put mine 1.5 points lower).

As a check of the method itself, I worked out LambdaMART's gradients by hand for one query, and they match LightGBM's.

5. Next steps (to Dec 8).

- **Tune both models like the paper:** LambdaMART's grid search; for DASALC, larger networks, longer training and noise on Kaggle GPUs, plus the paper's ensemble (the average of 3–5 runs). This should also show which empty-query rule the paper used.
- **Finish the comparison:** DASALC on Yahoo and Istella, significance tests, and which dataset traits (size, list length, number of features) favor trees or neural networks.
- **Extension:** add ColBERTv2 scores as LambdaMART features on Istella22, and compare three feature sets.
- **Cost:** prediction time per query for each model.